Assignment 2A — Prompting as System Design

Prompt patterns, reasoning chains and query routing

Overview

Build prompting patterns as plain Python functions, combine them into reasoning chains, then build a router that sends each query to the cheapest pattern that answers it well. Write each pattern yourself in plain Python, using only transformers and peft to run the model. Frameworks such as LangChain build prompts for you behind the scenes, and this assignment is about seeing exactly what each pattern sends to the model.

Use one notebook for the whole assignment.

Part	What you do	Marks
A	Build base prompting patterns and benchmark them with an LLM judge	8
B	Add reasoning patterns, ReAct and one composed chain	6
C	Build a router that picks a pattern for each query	6
Total		**20**

Setup (not graded)

- Load your Assignment 1 model, either your fine-tuned adapter (1A) or your 4-bit quantized model (1B), and wrap it as `generate(prompt) → text`.
- Take 10 queries with known answers from `instruction_dataset.jsonl` as your benchmark set. No new data collection or training is needed.
- Fix the decoding settings: greedy for 1A, or your best setting from 1B, with `max_new_tokens = 200`.
- Write every pattern as a Python function that takes a query and returns `(answer, tokens_used, latency_sec)`. `tokens_used` is prompt tokens plus generated tokens, summed over every LLM call the pattern makes.

Part A — Base patterns (8 marks)

Zero-shot sends the query as it is. Build it first: it is your baseline in every comparison, and it is not graded on its own.

A1. Few-shot and instruction prompts (2 marks)

1. **Few-shot:** put 3 worked examples from `instruction_dataset.jsonl` before the query. Do not use any of your 10 benchmark queries as examples. **(1 mark)**

2. **Instruction:** wrap the query in a template with a role and numbered steps, for example "You are a … Follow these steps: 1. … 2. … 3. …". Show your template. **(1 mark)**

A2. Self-critique (2 marks)

1. Make three LLM calls in sequence: answer the query, critique that answer, then revise the answer using the critique. Each call's output goes into the next call's prompt. **(1 mark)**

2. For 2 queries, print the first answer, the critique and the revised answer. Say in one sentence each whether the revision improved the answer. **(1 mark)**

A3. Structured output (2 marks)

1. Define a JSON schema for your domain, for example `{entity, value, unit, source_clause}`, and prompt the model to return only JSON matching it. Parse each output with `json.loads` and check that the required fields are present. Show 3 valid outputs. **(1 mark)**

2. Show one case where parsing fails and how you handle it, for example a retry prompt that includes the error message. Show the output after the retry. **(1 mark)**

A4. Benchmark with an LLM judge (2 marks)

An **LLM judge** is a second LLM that compares each answer with the known answer and gives it a score, like an examiner with a marking scheme. Use a judge model different from your generator, for example Llama 3.1 8B Instruct or Qwen2.5 7B Instruct, and name it. You will reuse this judge in Parts B and C.

Hints for a good judge prompt:

- Give the judge the query, the known answer and the pattern's answer.
- Define each score exactly: **2** = correct (all key facts, nothing wrong), **1** = partly correct (some key facts missing, nothing wrong), **0** = wrong.
- Ask for a one-sentence reason with each score.
- Fix the output format, for example `SCORE: 2` and `REASON: ...` on separate lines, so you can read the results with code.
- Set the temperature to 0, so the judge gives the same score every time.

1. Write your judge prompt and show it. Run all five patterns (zero-shot, few-shot, instruction, self-critique, structured output) on the 10 queries, score every answer with the judge, and fill this table. **(1 mark)**

Pattern	Average score (0–2)	Average tokens	Average latency (s)
Zero-shot			
Few-shot			
Instruction			
Self-critique			
Structured output			

2. Spot-check 5 of the judge's scores yourself and report how many you agree with. Then say in one or two sentences whether self-critique justifies its extra tokens over zero-shot. **(1 mark)**

Part B — Reasoning patterns and a chain (6 marks)

Write 2 multi-step problems from your domain, with known answers, that need more than one reasoning step to solve. Use them in B1 and B3, and score every answer with your judge from A4.

B1. Chain-of-thought and self-consistency (2 marks)

1. **Chain-of-thought:** prompt the model to reason step by step before giving its final answer. Run it on both problems and show the reasoning. **(1 mark)**

2. **Self-consistency:** sample 5 chain-of-thought answers to each problem with temperature above 0, and take the most common final answer. Show the 5 final answers and the vote. **(1 mark)**

Pattern	Problem 1 score (0–2)	Problem 2 score (0–2)	Average tokens
Zero-shot			
Chain-of-thought			
Self-consistency			

Score each cell with your judge from A4, comparing the pattern's final answer with the problem's known answer. Run zero-shot on both problems for the baseline row. For chain-of-thought, ask the model to end with Final answer: ... and judge only that line. For self-consistency, judge the majority-vote answer; if there's a tie, use the earliest sample from the largest group. Average tokens for self-consistency include all 5 samples.

B2. ReAct (2 marks)

ReAct repeats a loop of **Thought** (the model decides what to do), **Action** (it calls a tool) and **Observation** (the tool's result goes back into the prompt).

1. Write at least one local Python tool, for example a calculator or a lookup over a small dictionary, and a loop that reads the model's Action, runs the tool and adds the Observation to the next prompt. Stop when the model gives a final answer or after 5 cycles. **(1 mark)**
2. Run it on one domain question that needs the tool. Print the full trace, with at least 3 cycles, and the final answer. **(1 mark)**

B3. Plan → Solve → Verify chain (2 marks)

Compose patterns you have already built: one call writes a plan, one call solves the problem following the plan, and the self-critique from A2 checks and revises the answer. This suits high-stakes questions where a wrong answer is costly.

1. Implement the chain and show the plan, the solution and the verified answer for one problem. **(1 mark)**

2. Run it on both problems and add a row to the B1 table. In two or three sentences, say whether the chain beat your best single pattern, and whether that was worth the extra tokens. **(1 mark)**

Part C — Prompt router (6 marks)

A router sends each query to the cheapest pattern likely to answer it well. Use this routing table:

Query type	Route to
Simple factual question	Zero-shot
Structured extraction	Structured output (A3)
Multi-step reasoning	Chain-of-thought (B1)
Needs a calculation or tool	ReAct (B2)
High-stakes, needs checking	Plan → Solve → Verify (B3)

C1. Query classifier (2 marks)

1. Write a zero-shot classifier prompt that returns exactly one label from the table, with no explanation. Show the prompt. **(1 mark)**
2. Label 5 queries yourself, one per type, and test the classifier on them. Report its accuracy. If fewer than 4 of 5 are right, improve the prompt and test again, showing both versions. **(1 mark)**

C2. Router (2 marks)

1. Write a `route(query)` function that classifies the query, calls the chosen pattern, and returns `(answer, route, tokens_used, latency_sec)`. **(1 mark)**
2. Run the router on the 10 benchmark queries and score each answer with your judge from A4. Show a table with a row per query: query, route chosen, score, tokens. **(1 mark)**

C3. Cost against quality (2 marks)

1. Fill this table, reusing your zero-shot results from A4. **(1 mark)**

Approach	Average score (0–2)	Average tokens	Average latency (s)
Always zero-shot			
Router			

   Use the same judge prompt and the same 10 queries as A4, so the rows are comparable. Copy the always-zero-shot row from your A4 table. For the router row, average the 10 per-query scores from C2. Router tokens and latency must include the classifier call as well as the chosen pattern's calls.

2. Write about 100 words: which route did the router choose most often? On which queries did routing beat zero-shot, and was the extra token cost worth it there? Which single pattern would you deploy in production if you could not route, and why? **(1 mark)**

Submission checklist

Files

- Notebook (.ipynb) with all outputs visible
- HTML export of the same notebook

Part A

- A1: few-shot examples used and instruction template
- A2: first answer, critique and revised answer for 2 queries
- A3: JSON schema, 3 valid outputs, one parse failure and its retry
- A4: judge model name and judge prompt, benchmark table, spot-check result, self-critique verdict

Part B

- Your 2 multi-step problems with their known answers
- B1: chain-of-thought reasoning, the 5 self-consistency samples and vote, filled table
- B2: tool code and full ReAct trace with the final answer
- B3: plan, solution and verified answer for one problem; chain row in the table; whether the chain was worth it

Part C

- C1: classifier prompt (both versions if refined) and accuracy on 5 labelled queries
- C2: router results table for the 10 queries
- C3: router vs zero-shot table and 100-word analysis